

Projektarbeit

Modul 223 | 25.09.2026 | Marharyta Oberemok | INA24B

Problemstellung

Als Künstlerin, die Schmuck selber erstellt, bekomme ich immer Fragen, wie ich die einzelnen Schmuckstücke gemacht habe. Es ist mir zu aufwändig, allen Interessierten denselben Prozess immer zu zeigen. Ausserdem wünsche ich mir gerne einen verallgemeinerten Austausch mit den anderen Künstlern.

Projekt

- Domäne: Kunst und Buchungsverwaltung
- Name der Applikation: AllAroundJewelry
- Vision: AllAroundJewelry wird entwickelt, um die Schmuck- Macher sowie die Interessierte auf einer Plattform zu verbinden, welche den Austausch zwischen beiden erleichtert und fördert.

Projektplanung: 1. MVP Iteration

Die wichtigste funktionale Anforderung für die erste Iteration ist die Erstellung und Verwaltung von einzelnen Kursen. Dies umfasst das Erstellen von Kursen und die entsprechende Buchung.

Anforderungsanalyse

Funktionale Anforderungen (priorisiert)

- Benutzer können sich registrieren und einloggen
- Benutzer können zu Schmuckersteller werden
- Kurse können erstellt, bearbeitet und gelöscht werden
- Kurse können von Benutzer gebucht werden
- Benutzer können ihr Profil bearbeiten und ihre E-Mail-Adresse ändern
- Benutzer können ihr Passwort zurücksetzen

Qualitätsattribute

- Benutzerfreundlichkeit: Eine Buchung ist von der Kursübersicht aus in höchstens drei Klicks abgeschlossen.
- Skalierbarkeit: Unterstützung von mindestens 10.000 gleichzeitigen Benutzern.
- Sicherheit: Passwörter werden nur als bcrypt-Hash gespeichert; Kurse bearbeiten kann ausschliesslich der erstellende Schmuckersteller.
- Datenkonsistenz: Bei zwei gleichzeitigen Reservierungen des letzten freien Platzes wird genau eine bestätigt.
- Kompatibilität: Unterstützung aller gängigen Webbrowser und mobilen Geräte.

Benutzerrollen

- Benutzer: Kann alle öffentliche Kurse anschauen und buchen.

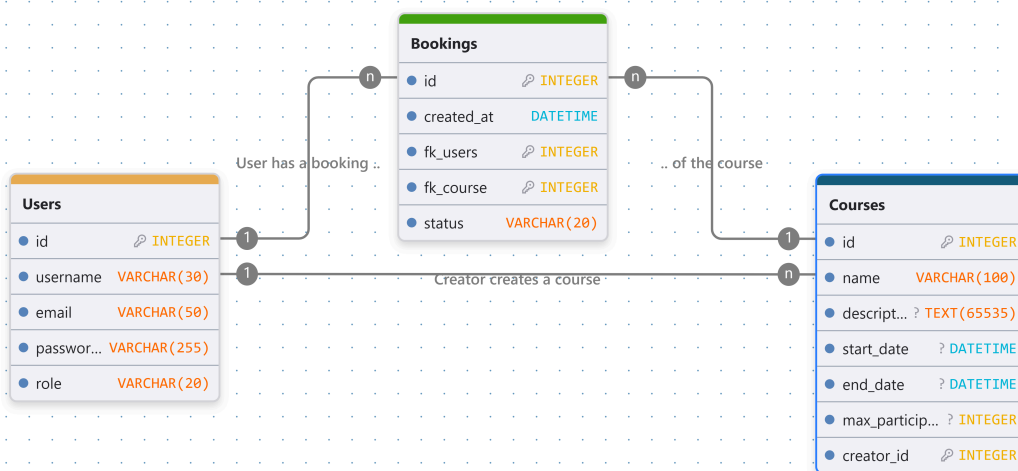
- Schmuckersteller: Kann eigene Kurse erstellen, bearbeiten und löschen, kann andere Kurse auch buchen.

Locking und Transaktionen

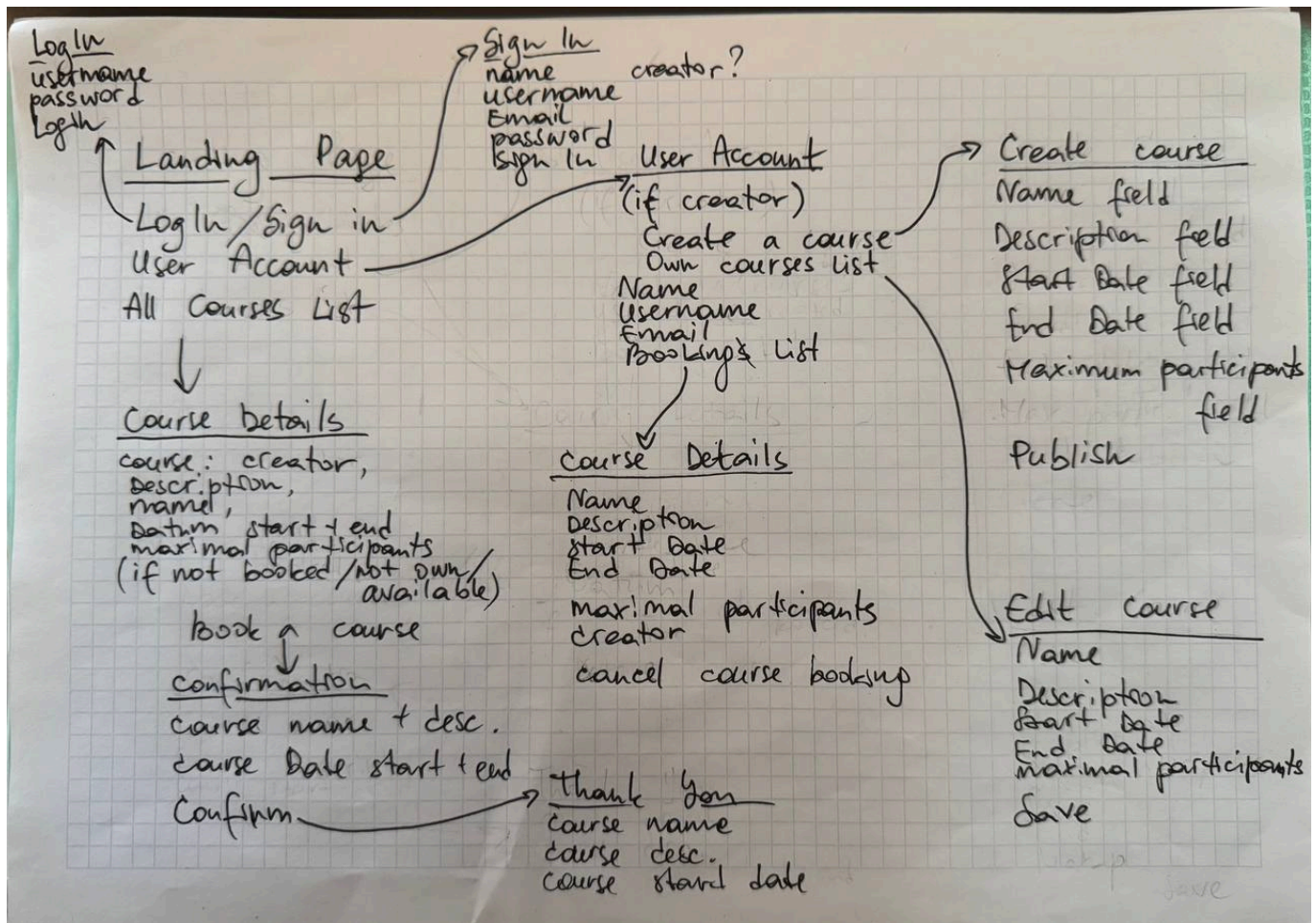
Kursbuchung: Beim Buchen von Kursen mit dem begrenzten Benutzeranzahl, muss geprüft werden, ob noch Plätze frei sind, und gleichzeitig die Buchung erstellt werden. Zwischen Prüfung und Speicherung kann ein anderer Benutzer denselben letzten Platz buchen (Race Condition). Deshalb wird der Kurs innerhalb einer Transaktion pessimistisch gesperrt (kurs.with_lock), damit die Platzzahl nie überschritten wird. Zusätzlich verhindert ein Unique Index auf (kurs_id, benutzer_id), dass jemand sich versehentlich doppelt anmeldet.

ERM (Entity-Relationship-Model)

fk_users und fk_course sind in der Tabelle Unique Index, users.role und booking.status sind enums.



Breadboards



Fat-Marker-Sketches



Die folgenden Abschnitte ergänzen die Dokumentation oben. Sie beschreiben, wie die erste Iteration umgesetzt wurde, wo und warum von der Planung abgewichen wurde, was noch offen ist und wie die Anforderungen geprüft wurden.

Technischer Rahmen

- Ruby on Rails 8.1, SQLite als Datenbank
- Eigene Authentifizierung (Rails-Generator): Session-Cookie, `Current.user`, Passwörter mit `has_secure_password` (bcrypt)
- Autorisierung mit Pundit (`CoursePolicy`, `BookingPolicy`, `UserPolicy`)
- Aktivitätsprotokoll mit dem Gem `audited` (Kurse und Buchungen)
- Tests mit Minitest (Model-, Policy-, Controller- und Integrationstests)

Rollen und Berechtigungen (umgesetzt)

Zusätzlich zu den geplanten Rollen gibt es den **Admin**. Der Admin ist keine eigene Rolle im Enum `users.role` (`user`, `creator`), sondern das Boolean-Feld `users.admin`.

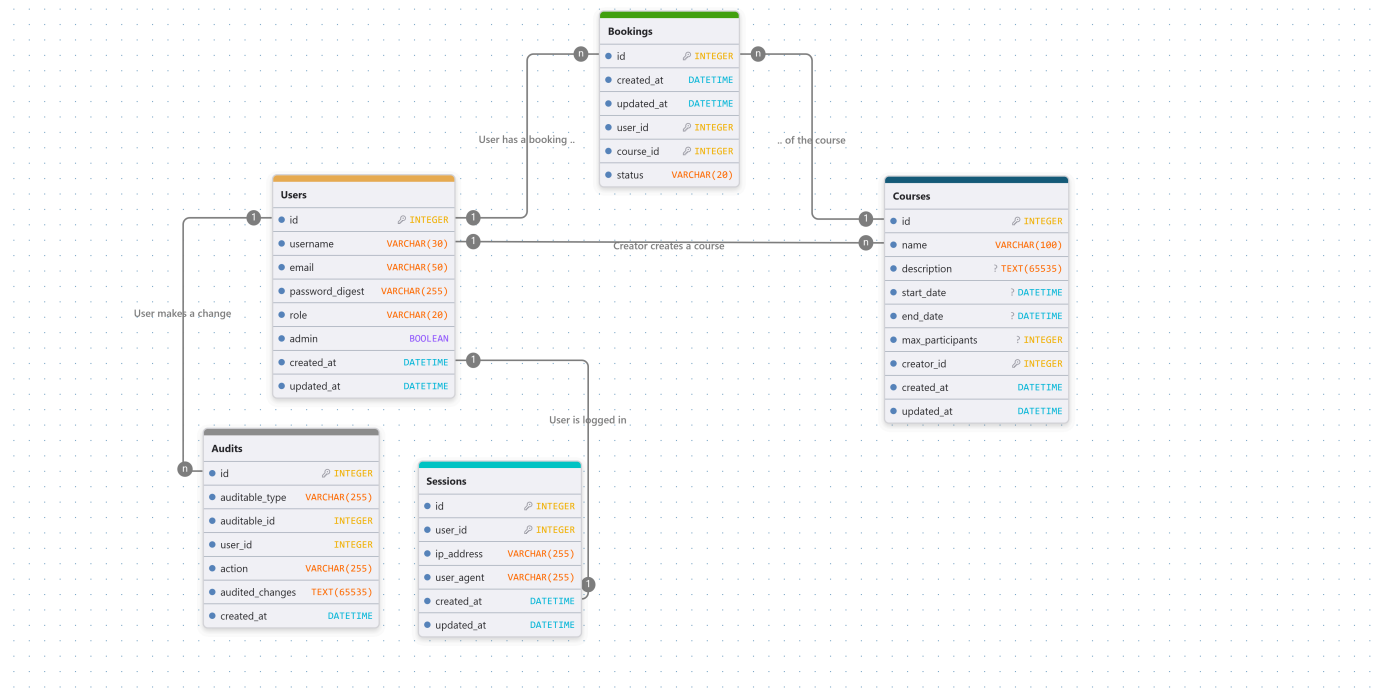
Aktion	Gast (nicht eingeloggt)	Benutzer	Schmuckersteller	Admin
Kursübersicht und Kursdetails ansehen	✓	✓	✓	✓
Kurs erstellen	✗	✗	✓	✓
Eigenen Kurs bearbeiten / löschen	✗	✗	✓	✓
Fremden Kurs bearbeiten / löschen	✗	✗	✗	✓
Kurs buchen (nicht voll, nicht bereits gebucht)	✗	✓	✓ (nicht den eigenen)	✓
Eigene Buchung ansehen	✗	✓	✓	✓
Fremde Buchung ansehen	✗	✗	✗	✓
Eigene bestätigte Buchung stornieren	✗	✓	✓	✓
Profil bearbeiten (Benutzername, E-Mail)	✗	✓	✓	✓
Benutzerverwaltung und Aktivitätsprotokoll (<code>/admin/users</code>)	✗	✗	✗	✓
Benutzer löschen	✗	✗	✗	✓ (nicht sich selbst)

Durchsetzung:

- **before_action :require_authentication** im **ApplicationController** leitet Gäste zur Anmeldung um. Ausnahmen sind die Kursübersicht und die Kursdetails, Login, Registrierung und das Zurücksetzen des Passworts.
- Jede schützenswerte Controller-Action ruft **authorize** auf. Die Regeln stehen in den Pundit-Policies.
- Ein verweigerter Zugriff (**Pundit::NotAuthorizedError**) führt zu einer Weiterleitung zur Startseite mit der Meldung „Kein Zugriff.“.
- Die Ansicht blendet Knöpfe nur aus (z. B. „Bearbeiten“ nur für den Besitzer). Geschützt wird serverseitig.

ERM – aktueller Stand der Umsetzung

Das ursprüngliche ERM müsste ein paar Erweiterungen haben:



Unterschiede zum geplanten ERM v2: Hauptsächlich sind zu fast jeder Tabelle c **reated_at** und **updated_at** hinzugefügt, **fk_users** und **fk_courses** sind nach **user_id** und **course_id** umgetauft (Rails-Namenskonvention) und zur Tabelle **Users** ist das Boolische Wert **admin** hinzugefügt.

Es sind auch zwei neue Tabellen entstanden, nämlich Tabelle **Audits** für das Aktivitätsprotokol und Tabelle **Sessions** für das Login.

Breadboards und Screens der ersten Iteration

Die Seiten aus dem Breadboard (**breadboard_v2.jpg**) sind so umgesetzt:

Breadboard	Route	Controller#Action
Landing Page / All Courses List	/ bzw. /courses	courses#index
Log In	/session/new	sessions#new
Sign In (Registrierung)	/users/new	users#new
Course Details + „book a course“	/courses/:id	courses#show

Breadboard	Route	Controller#Action
Confirmation + „Confirm“	/courses/:id/booking/new	bookings#new → bookings#create
Thank You	/bookings/:id	bookings#show
User Account (Bookings List, Own courses list)	/account	accounts#show
Course Details + „cancel course booking“	/account (Knopf „Stornieren“) bzw. /bookings/:id	bookings#destroy
Create course / Publish	/courses/new	courses#new / #create
Edit course / Save	/courses/:id/edit	courses#edit / #update
(zusätzlich) Profil bearbeiten	/account/edit	accounts#edit / #update
(zusätzlich) Passwort vergessen	/passwords/new	passwords#new
(zusätzlich) Benutzerverwaltung / Protokoll	/admin/users	admin/users#index

Die Gestaltung orientiert sich an den Design-Entwürfen im Ordner `designs/`.

Screens der Umsetzung

Startseite (Gast und eingeloggt)

HANDGEMACHTER SCHMUCK

Lerne Schmuck *selbst* zu erschaffen.

AllAroundJewelry verbindet Schmuck-Macherinnen und Neugierige auf einer Plattform. Entdecke Kurse für Perlenschmuck, Drahtwickeln, Harzkunst und Makramee — online und offline in der Schweiz.

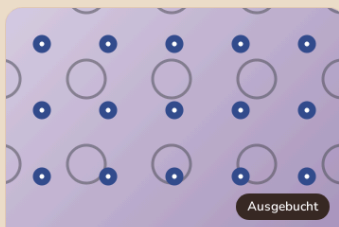
Kurse entdecken →

Konto erstellen

Aktuelle Kurse

Handverlesene Workshops von erfahrenen Schmuck-Macherinnen

Alle Kurse →



Ausgebucht

Perlenketten

Ketten knüpfen

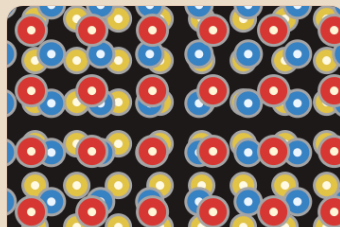
Montag, 5. Oktober 2026

06:33 · 1 Tag

creator

Details →

Ausgebucht



Ohrringe

Ohrstecker gestalten

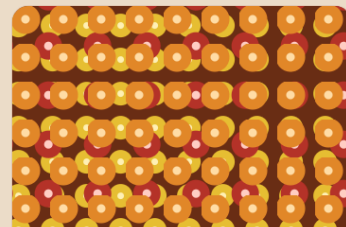
Montag, 5. Oktober 2026

06:33 · 1 Tag

creator

Details →

1/1 frei



Ringe schmieden

Silberringe selbst herstellen

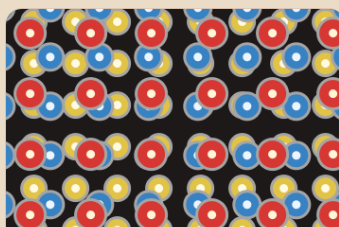
Montag, 5. Oktober 2026

06:33 · 1 Tag

creator

Details →

9/10 frei



Armbänder flechten

Lederarmbänder

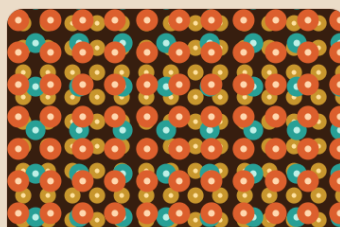
Donnerstag, 15. Oktober 2026

06:33 · 1 Tag

creator_two

Details →

5/5 frei



Goldschmieden Grundkurs

Einführung

Donnerstag, 15. Oktober 2026

06:33 · 1 Tag

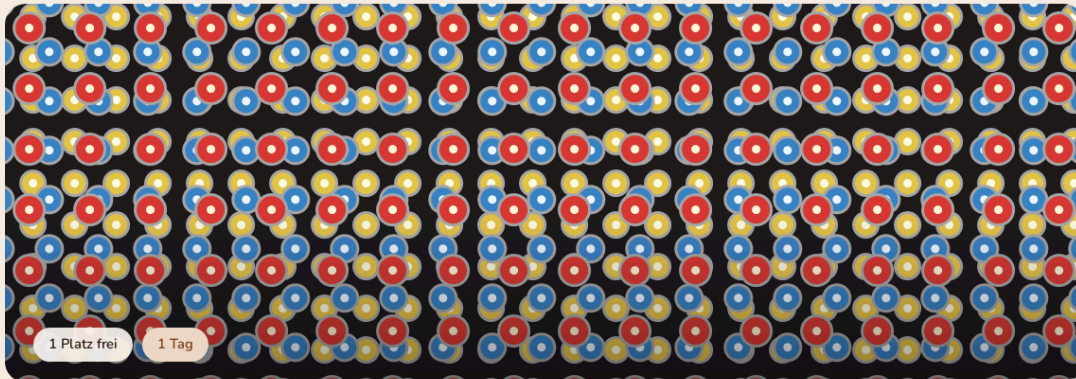
creator

Details →

5/5 frei

Kursdetails: ausgebucht und buchbar

Startseite › Kurse › Ohrringe



Ohrringe

**von creator**
Kursleitung

Kursbeschreibung

Ohrstecker gestalten

1 Platz frei

von 1 Plätzen · Anmeldung verbindlich



Datum

Montag, 5. Oktober 2026



Uhrzeit

06:33 Uhr



Dauer

1 Tag



Ende

06.10.2026, 06:33 Uhr



Plätze

1 von 1 frei

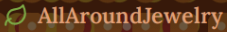
0 Angemeldet

Max. 1

Kurs buchen

[← Zurück](#)

Anmelden, Registrieren, Passwort vergessen



„Jede Perle erzählt eine Geschichte –
beginne deine heute.“

— Die AllAroundJewelry Community

[← Zurück zu den Kursen](#)

Willkommen zurück!

Melde dich an, um Kurse zu buchen und auf dein Profil zuzugreifen.

Benutzername

Passwort [Vergessen?](#)

Anmelden

Noch kein Konto? [Jetzt registrieren](#)

Kurs erstellen

AllAroundJewelry

Kurse

C creator ▾

Startseite › Kurse › Neuer Kurs

Neuen Kurs erstellen

Teile dein Wissen mit der Community.

Kursname

Beschreibung

Start

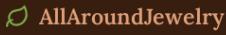
Ende

Maximale Teilnehmerzahl

Kurs erstellen

[Abbrechen](#)

Muss nach dem Start liegen.

AllAroundJewelry

Kurse Mein Konto

© 2026 AllAroundJewelry

Admin: Benutzerverwaltung und Aktivitätsprotokoll

AllAroundJewelry

Kurse A admin

Alle Benutzer

Verwaltung der Konten und Aktivitätsprotokoll

USERNAME	E-MAIL	ROLLE	
C creator_two	creator_two@example.com	creator	Löschen
A admin	admin@example.com	user admin	
U user	user@example.com	user	Löschen
C creator	creator@example.com	creator	Löschen

Aktivitätsprotokoll

Noch keine Aktivitäten.

Locking und Transaktionen, Umsetzung

Das Konzept oben ist in `app/models/booking_service.rb` umgesetzt:

```
def self.book!(course, user)
  Course.transaction do
    course.lock! # Kurs pessimistisch sperren
    raise CourseFull if course.full? # Prüfung innerhalb der Sperre

    existing = Booking.find_by(course: course, user: user)
    if existing
      existing.update!(status: "confirmed") # Falls storniert war, neu buchen
      existing
    else
      Booking.create!(course: course, user: user, status: "confirmed")
    end
  end
end
```

- **Transaktion:** Die Prüfung der freien Plätze und das Speichern der Buchung passieren in einer Transaktion. Wirft `CourseFull` eine Ausnahme, wird alles zurückgerollt und keine Buchung gespeichert. Der `BookingsController` fängt die Ausnahme ab und zeigt „Der Kurs ist inzwischen ausgebucht.“.
- **Pessimistisches Locking:** Statt `kurs.with_lock` wird `course.lock!` innerhalb von `Course.transaction` verwendet.

- **Doppelbuchung:** Zwei Ebenen verhindern, dass sich jemand doppelt anmeldet:
 1. Die Validierung in **Booking** erlaubt pro Benutzer und Kurs nur eine *bestätigte* Buchung.
 2. Der Unique Index auf **bookings** (*user_id*, *course_id*) sichert das zusätzlich auf Datenbankebene ab.
- **Stornierung und Neubuchung:** Beim Stornieren wird die Buchung nicht gelöscht, sondern auf *cancelled* gesetzt. Der Unique Index gilt für alle Status, deshalb wird bei einer Neubuchung die bestehende Zeile wieder auf *confirmed* gesetzt, statt eine zweite anzulegen.

Erreichter Stand

Funktionale Anforderungen

#	Anforderung	Stand
1	Benutzer können sich registrieren und einloggen	✓ (inkl. Logout)
2	Benutzer können zu Schmuckersteller werden	+ - teilweise: Die Rolle wird bei der Registrierung gewählt, später lässt sie sich nicht mehr ändern
3	Kurse können erstellt, bearbeitet und gelöscht werden	✓ (nur Besitzer und Admin)
4	Kurse können von Benutzer gebucht werden	✓
5	Benutzer können ihr Profil bearbeiten und ihre E-Mail-Adresse ändern	✓
6	Benutzer können ihr Passwort zurücksetzen	✓ (E-Mail mit Link in Terminal)

Zusätzlich umgesetzt:

- Admin Bereich mit Benutzerverwaltung
- Aktivitätsprotokoll: Wer hat wann einen Kurs oder eine Buchung erstellt, geändert oder gelöscht
- Stornierung von Buchungen
- Anzeige der freien Plätze mit Belegungsbalken
- Automatisierte Tests

Begründete Abweichungen

1. **Admin zusätzlich zu Benutzer und Schmuckersteller:** Für die Vereinheitlichung soll der Admin nicht als Boolean-Feld gesetzt werden, sondern als dritter Enum-Wert, weil Admin eine weitere Rolle ist.
2. **Registrierung ohne separates Feld „Name“:** Das Breadboard sah „name“ und „username“ vor. Umgesetzt ist nur *username*, weil das Modell kein Namensfeld hat.
3. **Spaltennamen im ERM:** *user_id* und *course_id* statt *fk_users* und *fk_course*, sowie andere Erweiterungen an der ERM (siehe ERM).
4. **Kursbilder als Platzhalter:** Die Design-Entwürfe zeigen Fotos. Weil es noch keinen Bild-Upload gibt, zeigen die Kurse Platzhalter-Muster in CSS.

Offene Punkte

Aus den Design-Entwürfen, im Modell noch nicht vorhanden:

- Format und Ort (Online / Offline, z. B. „Atelier in Bern“) sowie der Meeting-Link sollten für mehr Infos hinzugefügt werden und könnten eigene Zeilen in der Tabelle **Courses** haben.
- Niveau (z. B. „Anfänger“), Kategorie (Perlen, Draht, Harz ...) und Schlagwörter würden für die Einstufung helfen
- Kursbilder Hochladen wäre eine gute Ergänzung
- Vollständiger Name und Kurzbiografie der Kursleitung für Creators
- „Passwort bestätigen“ bei der Registrierung

Funktion und Sicherheit:

- **Rolle nicht änderbar:** Ein Benutzer kann nach der Registrierung nicht mehr zum Schmuckersteller werden.
- **Kursdaten:** Kurse in der Vergangenheit können noch gebucht werden. Start- und Enddatum sind keine Pflichtfelder.
- **Benutzer löschen:** Löscht ein Admin einen Schmuckersteller, werden dessen Kurse samt den Buchungen anderer Teilnehmender mitgelöscht, ohne dass diese benachrichtigt werden.

Qualität und Betrieb:

- **Nebenläufigkeit:** Die Tests prüfen die Buchungslogik nur nacheinander. Ein echter Test mit gleichzeitigen Threads fehlt.

Prüfung der Anforderungen und Ergebnisse

Funktionale Anforderungen

Anforderung	Prüfung	Ergebnis
Registrieren	UserControllerTest : erfolgreich, doppelte E-Mail, fehlendes Passwort; AuthenticationTest : doppelter Benutzername	✓
Einloggen / Ausloggen	SessionsControllerTest : richtige und falsche Daten, Logout; AuthenticationTest : unbekannter Benutzer	✓
Zu Schmuckersteller werden	UserControllerTest „create als creator“; AccountsControllerTest „update ignoriert role und admin“	🕒 nur bei der Registrierung möglich
Kurse erstellen / bearbeiten / löschen	CoursesControllerTest (27 Tests), CoursePolicyTest	✓
Kurse buchen	BookingsControllerTest , BookingsFlowTest , BookingServiceTest , BookingPolicyTest	✓
Buchung stornieren	BookingsControllerTest „destroy ...“	✓
Profil und E-Mail bearbeiten	AccountsControllerTest „update ändert eigenes Profil“, ungültige Daten	✓

Anforderung	Prüfung	Ergebnis
Passwort zurücksetzen	<code>PasswordsControllerTest</code> (7 Tests)	✓

Qualitätsattribute

Qualitätsattribut	Prüfung	Ergebnis
Benutzerfreundlichkeit: Buchung in höchstens drei Klicks	Manuell eingeloggt: Kursübersicht → Kurs (1) → „Kurs buchen“ (2) → „Verbindlich buchen“ (3)	✓
Skalierbarkeit: 10'000 gleichzeitige Benutzer	Nicht geprüft	✗
Sicherheit: Passwörter als bcrypt-Hash	<code>has_secure_password</code> ; in der Datenbank steht nur <code>password_digest</code>	✓
Sicherheit: Nur der Ersteller bearbeitet Kurse	<code>CoursePolicyTest</code> ; <code>AuthorizationTest</code> (PATCH/DELETE auf fremden Kurs als User und Gast); <code>CoursesControllerTest</code> (als fremder Schmuckersteller)	✓ (Admin darf auch)
Datenkonsistenz: Letzter Platz wird genau einmal vergeben	<code>BookingServiceTest</code> : letzter Platz, zweite Person erhält <code>CourseFull</code> , bei <code>CourseFull</code> keine Buchung in der DB	🕒 Logik und Rollback bestanden; gleichzeitige Zugriffe nicht mit Threads getestet
Kompatibilität: Browser und mobile Geräte	Screenshots in Chromium bei 1440 px und 400 px; responsives Layout	🕒 Chromium geprüft; andere Browser nicht getestet, alte Browser durch <code>allow_browser</code> gesperrt

Berechtigungen und Schutz vor direkten Requests

Prüfung	Test	Ergebnis
PATCH / DELETE auf fremden Kurs	<code>CoursesControllerTest</code> (fremder Schmuckersteller), <code>AuthorizationTest</code> (User, Gast)	✓ verweigert
DELETE auf fremde Buchung	<code>BookingsControllerTest</code> (fremder Schmuckersteller), <code>AuthorizationTest</code> (Admin)	✓ verweigert
GET <code>/admin/users</code> als Nicht-Admin	<code>AuthorizationTest</code> (User, beide Schmuckersteller), <code>Admin::UsersControllerTest</code> (Gast)	✓ verweigert
Admin löscht sich selbst	<code>Admin::UsersControllerTest</code> , <code>UserPolicyTest</code>	✓ verweigert

Locking, Transaktionen und Aktivitätsprotokoll

Prüfung	Test	Ergebnis
Bei <code>CourseFull</code> landet keine Buchung in der DB	<code>BookingServiceTest</code> „bei CourseFull wird keine Buchung gespeichert“	
Stornierte Buchung bleibt bei vollem Kurs storniert	<code>BookingServiceTest</code>	
Doppelte bestätigte Buchung wird abgelehnt	<code>BookingTest</code>	
Audit bei Kurs-Erstellung, Buchung und Stornieren mit gesetztem <code>audit.user</code>	<code>AuditLogTest</code>	